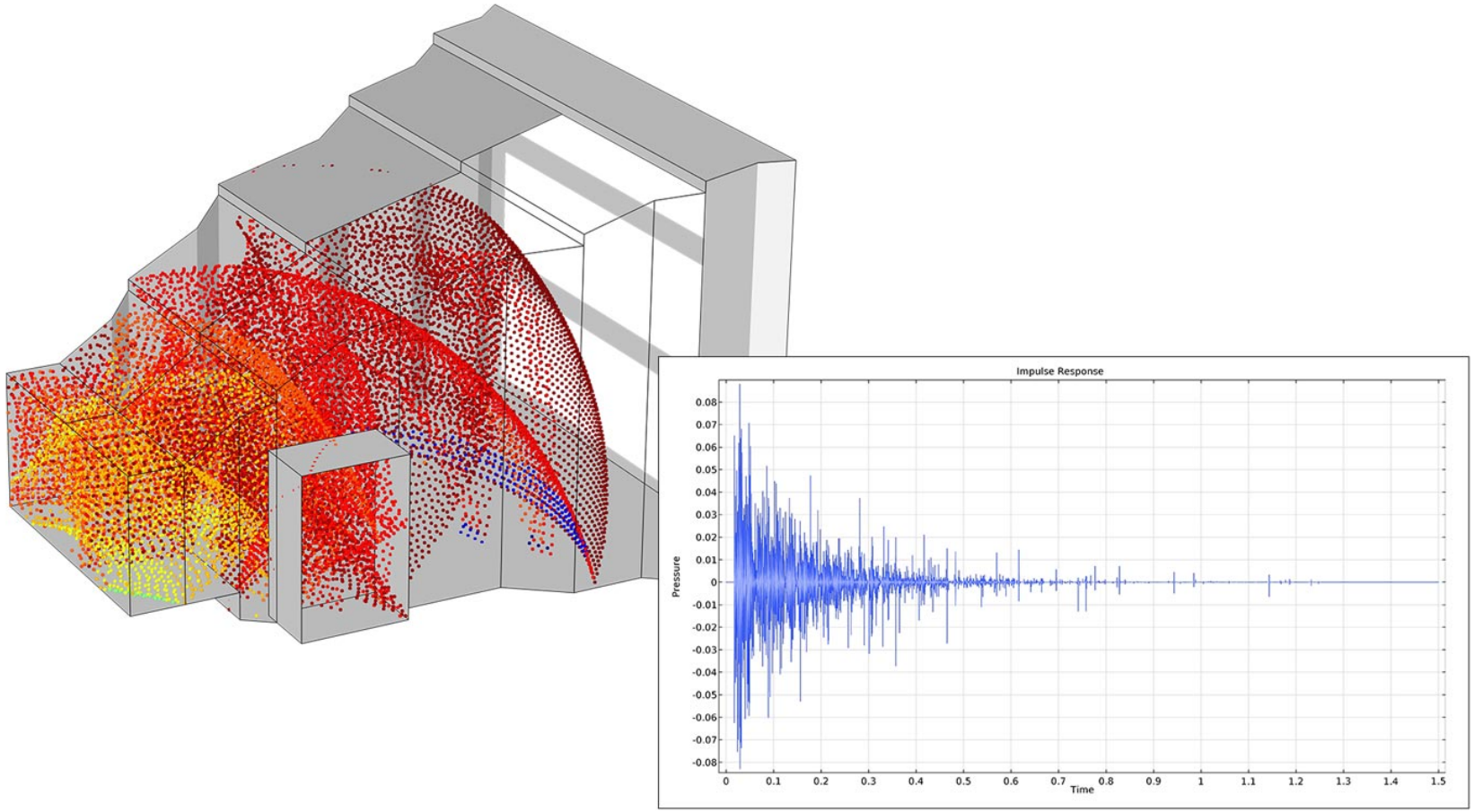
# Jet marching methods for numerical geometric acoustics

Samuel F. Potter
Joint work with Dr. Maria Cameron

UMD Dept. of Computer Science

January 8, 2021

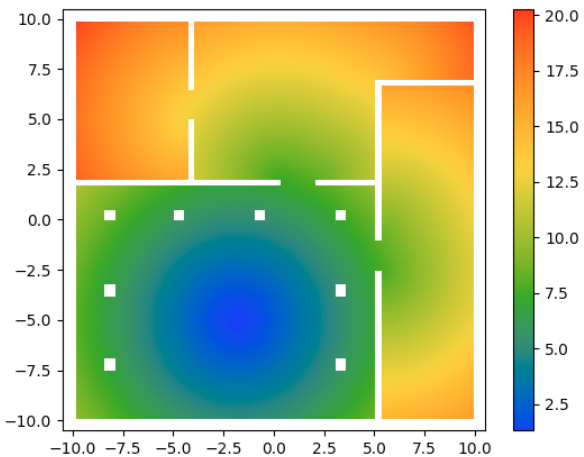
# Computational room acoustics

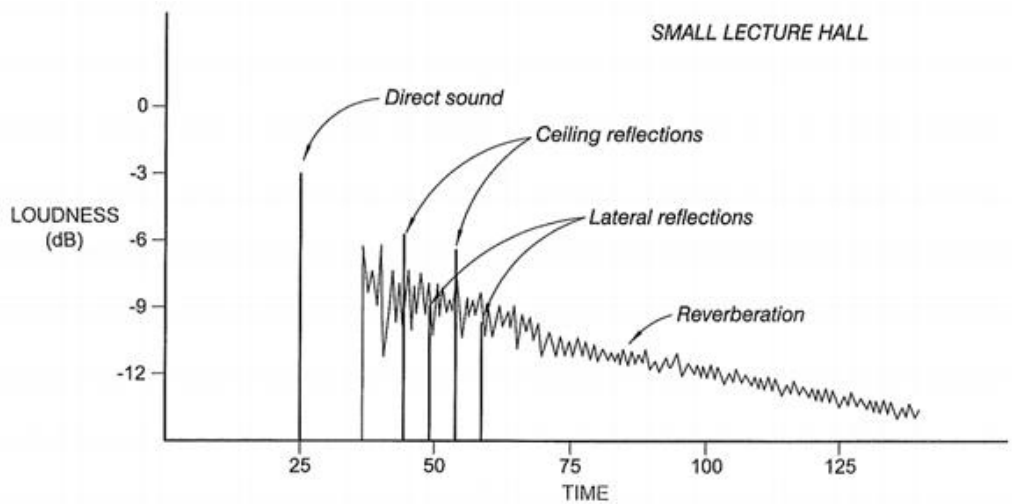



Source: `comsol.com`

# Precomputed sound propagation

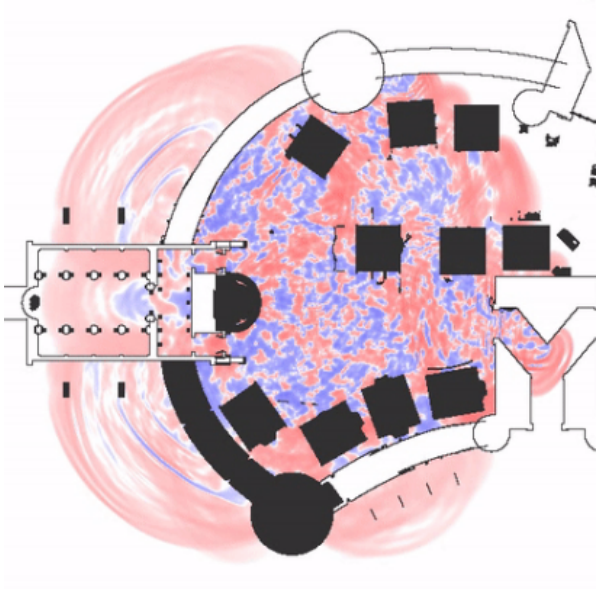
**Basic task:** for a pair of points in the room (a *source* and *listener*), compute or encode the room impulse response. This is the Green's function for the wave equation for a *fixed sound source location*.

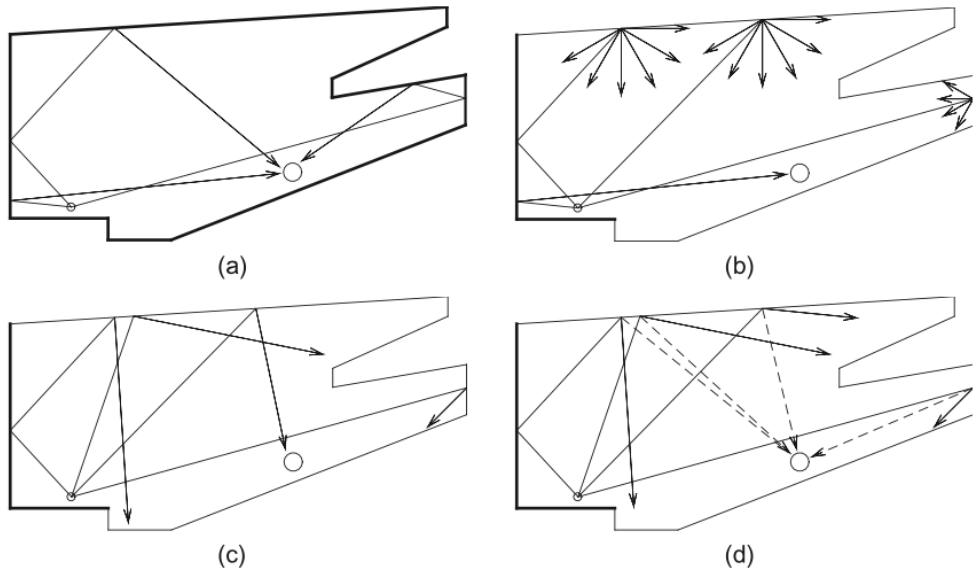



Source: `https://sites.google.com/site/roomacgroup4/`

# Numerical vs. geometric acoustics






**Numerical acoustics**: solve wave equation for bandlimited impulse input. Eulerian.

**Geometric acoustics**: use raytracing to assemble room impulse response. Lagrangian.

---

Source: `https://www.microsoft.com/en-us/research/project/project-triton`

# Geometric optics from Helmholtz

For sound speed $c$ and freq. $\omega \gg 0$, the Helmholtz equation is:

$$\left(\Delta + \frac{\omega^2}{c^2}\right) U = 0$$

High-frequency geometric optics ansatz (WKB) is:

$$U(x) = \alpha(x) e^{i\omega\tau(x)} + O(\omega^{-1})$$

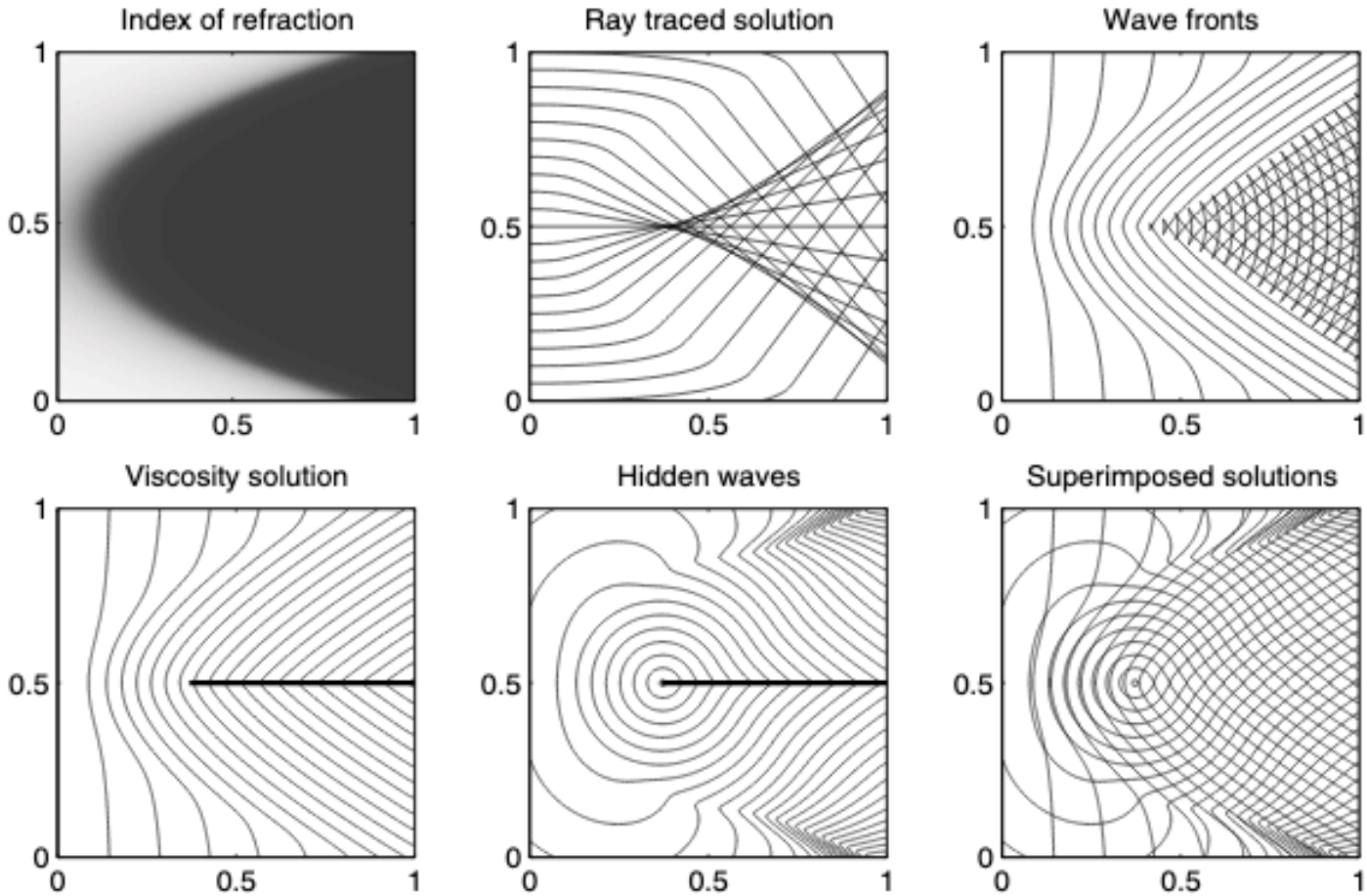
Here $\tau$ satisfies eikonal equation:

$$\|\nabla\tau\| = 1/c$$

and $\alpha$ is amplitude, which satisfies the transport equation:

$$\alpha\Delta\tau + 2\nabla\alpha^\top\nabla\tau = 0$$

# “Numerical geometric acoustics”




**Geometric acoustics using semi-Lagrangian numerical algorithms** (“Eulerian geometric optics”)

Source: *“Computational high frequency wave propagation”*, Engquist and Runborg.

# Jet marching methods for solving the eikonal equation

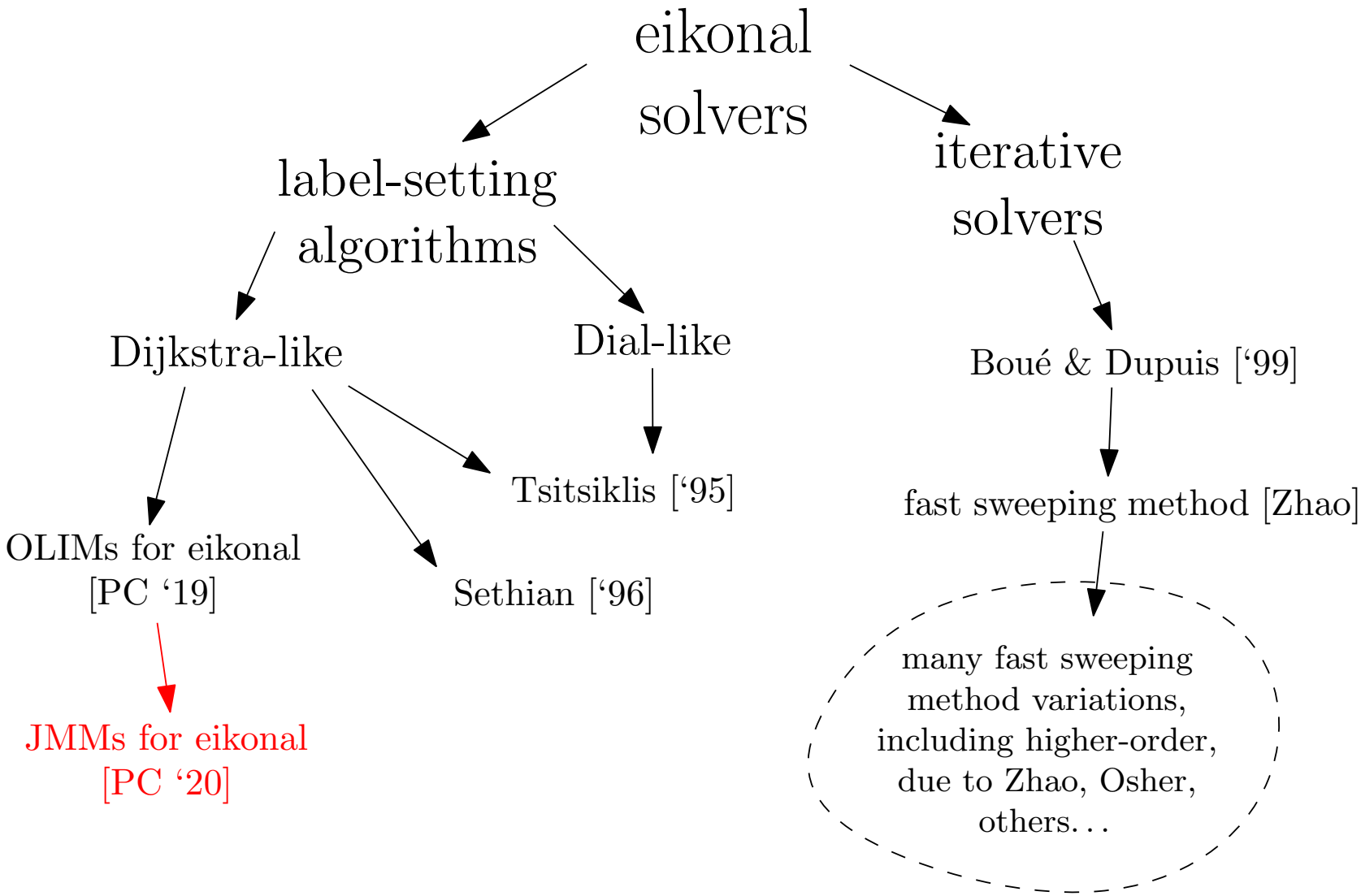
# Desiderata

We need an eikonal solver that:

- Is at least $O(h^{3-p})$ accurate for $\partial^\alpha \tau$, $|\alpha| = p$
- Has a *compact stencil*
- Parametrizes local characteristics
- Is causal/is a direct solver
- Runs in optimal time ($O(N)$ or $O(N \log N)$ time)

Key idea: **design solver based on Hermite interpolation, semi-Lagrangian updates, and cell marching**

# Prior art


eikonal solvers
label-setting algorithms
iterative solvers
Dijkstra-like
Dial-like
Boué & Dupuis [‘99]
Tsitsiklis [‘95]
fast sweeping method [Zhao]
OLIMs for eikonal [PC ‘19]
Sethian [‘96]
many fast sweeping method variations, including higher-order, due to Zhao, Osher, others...
JMMs for eikonal [PC ‘20]

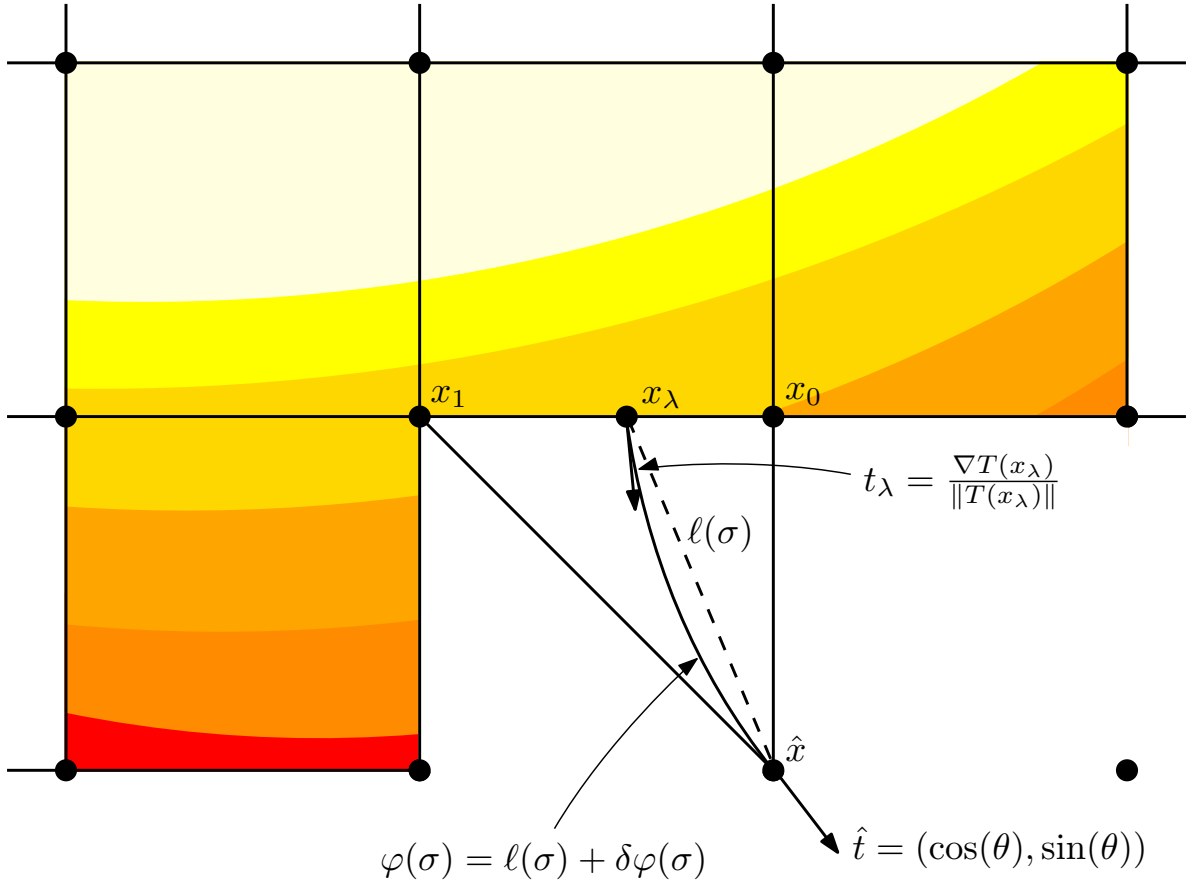
# "Dijkstra-like" solvers

Roughly:

- Discretize domain $\Omega$ to get $\Omega_h$
- Use states $\{\texttt{far}, \texttt{trial}, \texttt{valid}\}$ to label nodes in $\Omega_h$
- Initialize $T(x)$ with boundary conditions
- Use a priority queue (binary heap) to order `trial` nodes by $T$, and:
  1. Pop a node from the heap
  2. Set state to `valid`
  3. `far` neighbors $\rightarrow$ `trial`
  4. "Update all `trial` neighbors"

**Dial-like solvers**: use a bucketed priority queue $\rightarrow$ relax many nodes simultaneously $\rightarrow$ $O(N)$ complexity + parallelism instead of just $O(N \log N)$

# Problem setup



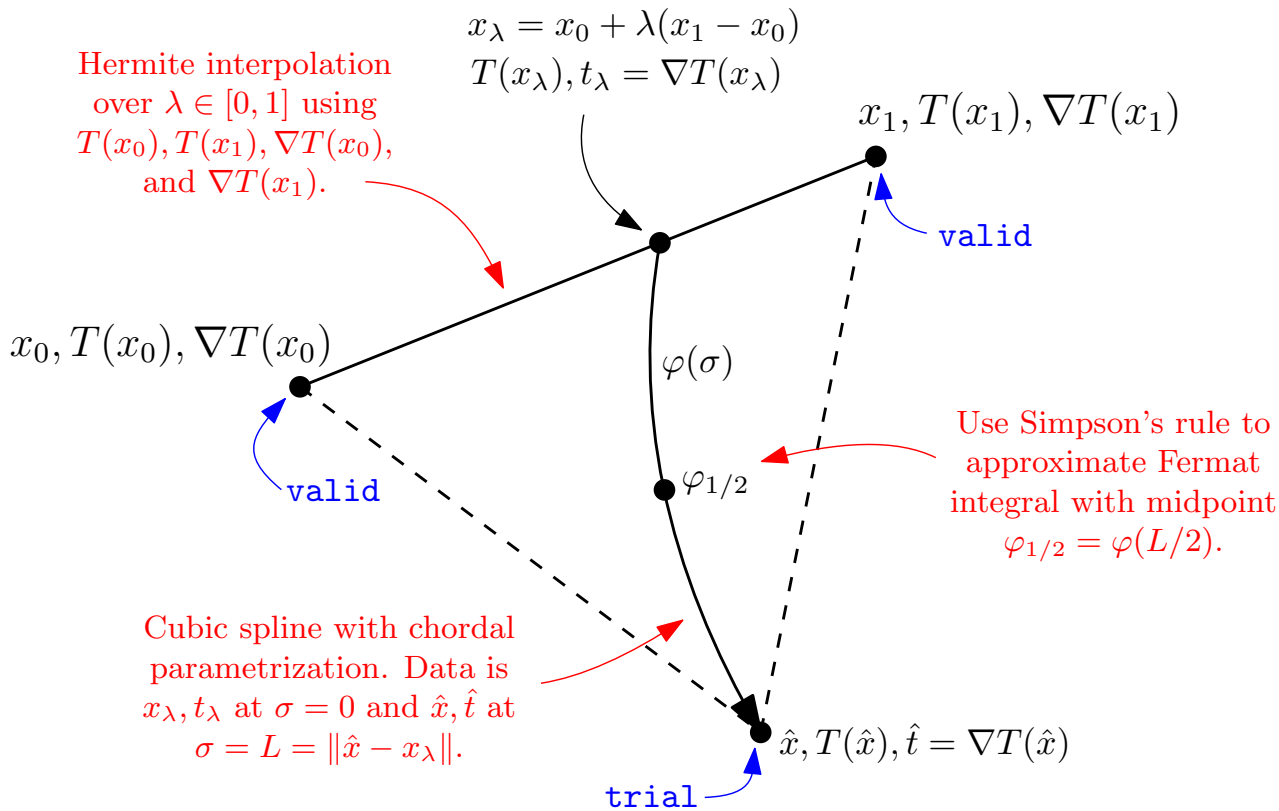

Fermat's principle:

$$\tau(\hat{x}) = \min_{x,\psi} \left\{ \tau(x) + \int_0^1 s(\psi(\sigma)) \|\psi'(\sigma)\| d\sigma \right\}$$

---

Nice video here: `https://youtu.be/Ze9AeDbaDVM`

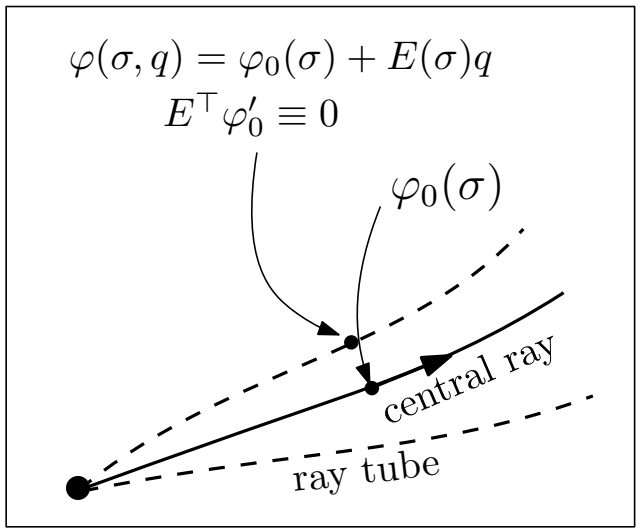
# Doing updates in 2D




$$T(\hat{x}) = \min_{\substack{\hat{t}\in\mathbb{S}^1 \\ 0\le\lambda\le 1}} \left\{ \mathsf{T}(\lambda) + \frac{L}{6}\left( s(\hat{x}) + 4s\left(\varphi\left(\tfrac{L}{2}\right)\right)\left\|\varphi'\left(\tfrac{L}{2}\right)\right\| + s(x_\lambda)\right)\right\}$$

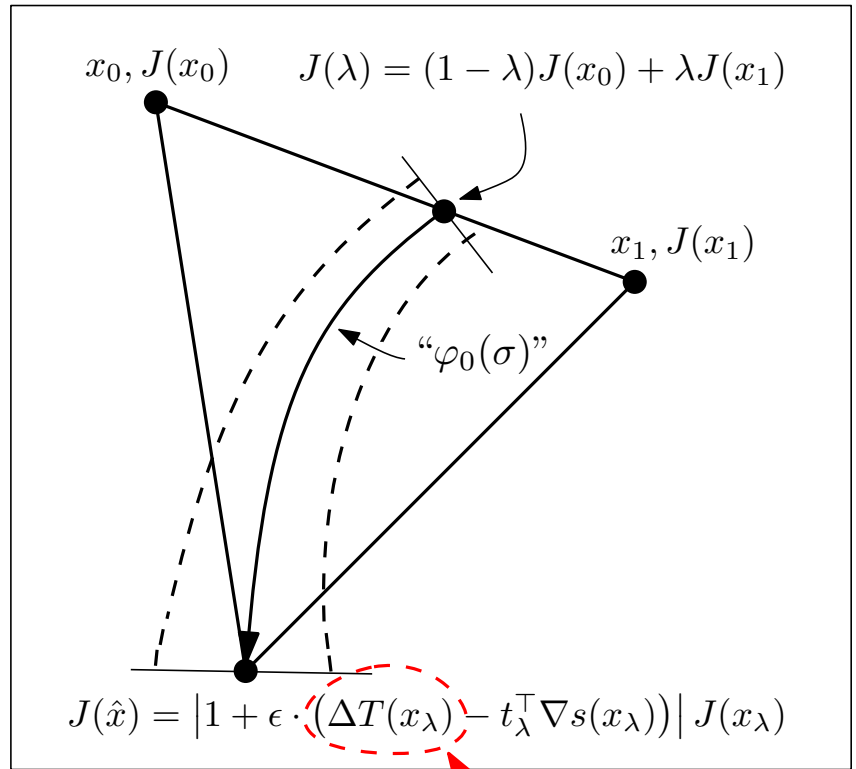
Source: "Jet marching methods for solving the eikonal equation", Potter and Cameron, *in submission*.

# Marching the amplitude: local paraxial ray theory




$$\begin{bmatrix} \dot{Q} \\ \dot{P} \end{bmatrix} = \begin{bmatrix} & c_0 I \\ \frac{-1}{c_0} \frac{\partial^2 c}{\partial q \partial q^\top}\Big|_{q=0} & \end{bmatrix} \begin{bmatrix} Q \\ P \end{bmatrix}$$

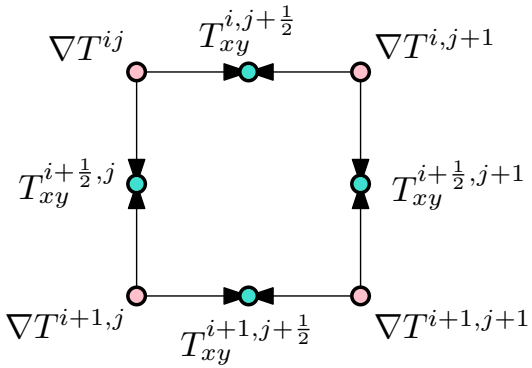
$$J = |\det Q|, \quad A(\phi_0(\sigma)) = \sqrt{\tfrac{c(\phi_0(\sigma))}{J(\phi_0(\sigma))}} A(\phi_0(0))$$



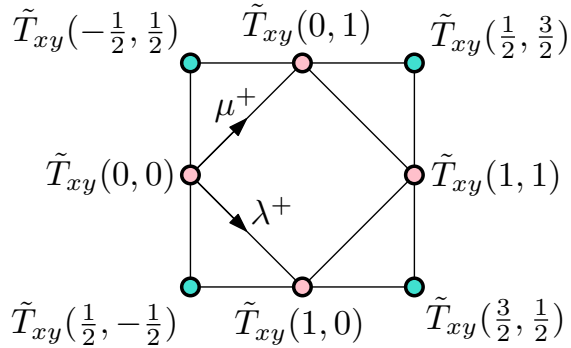

Source: "Computation of ray amplitudes in inhomogeneous media with curved interfaces", Popov, Pšenčík, Červený, 1978.

# Cell marching

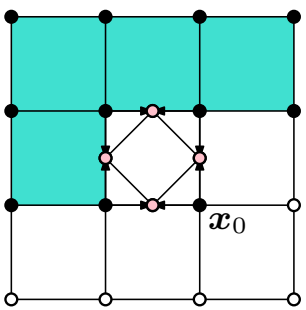
## One way to march second partials:



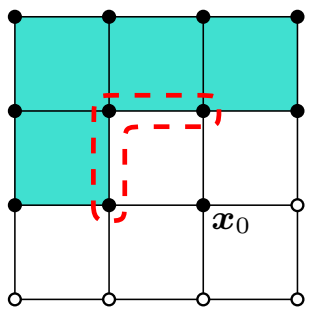

approximate $T_{xy}$ at cell edge midpoints using central differences



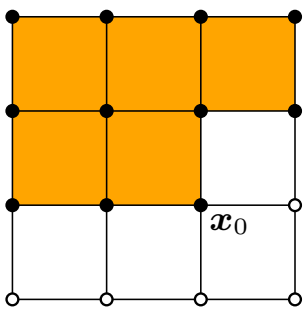

approximate $T_{xy}$ at cell vertices using bilinear extrapolation



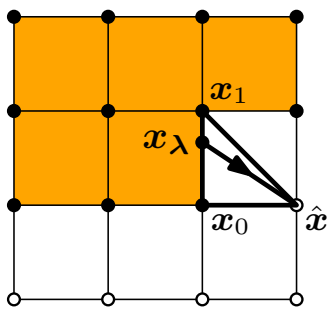

compute new $T_{xy}$ values at corners of new `valid` cell




average $T_{xy}$ values over adjacent `valid` cells

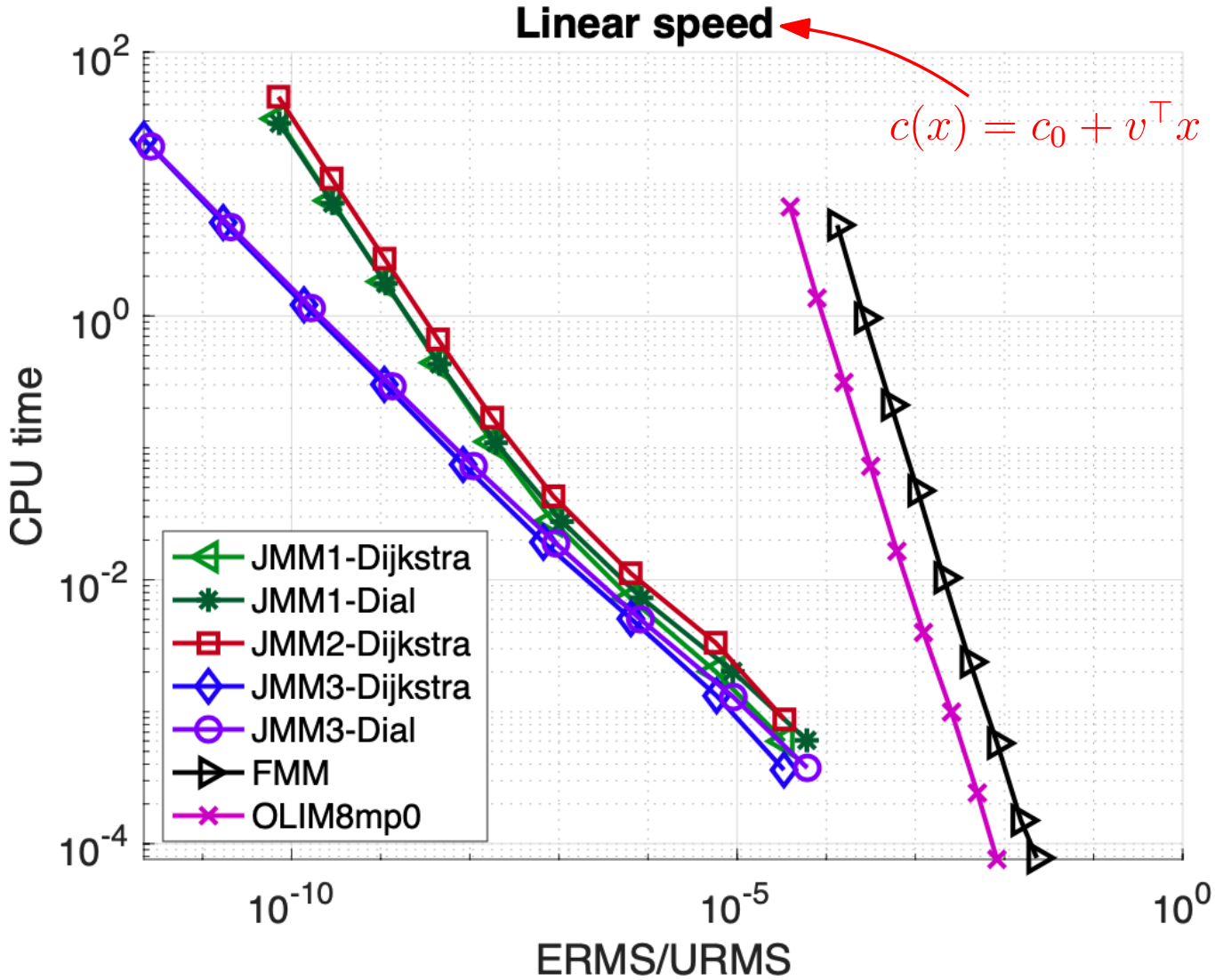



rebuild bicubics on affected `valid` cells




update neighboring `trial` nodes using triangle updates

# Numerical results



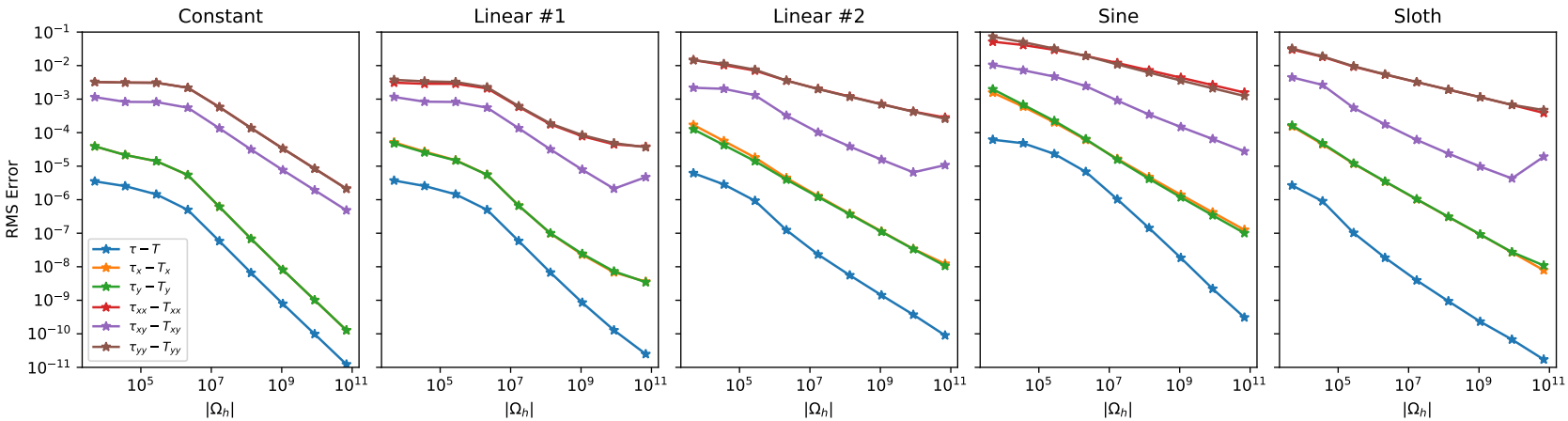
# Numerical results

The order of convergence $p$ for each combination of test problems and solvers, computed for different types of errors and fit as $Ch^p$.

| | JMM | $E_{\max}(T)$ | $E_{\mathrm{RMS}}(T)$ | $E_{\max}(\nabla T)$ | $E_{\mathrm{RMS}}(\nabla T)$ |
|---|---|---|---|---|---|
| Constant | #1 | 2.87 | 2.87 | 2.28 | 2.72 |
| | #2 | 2.87 | 2.87 | 2.28 | 2.72 |
| | #3 | 2.87 | 2.87 | 2.28 | 2.72 |
| Linear #1 | #1 | 2.77 | 2.85 | 2.14 | 2.52 |
| | #2 | 2.77 | 2.85 | 1.70 | 2.48 |
| | #3 | 2.86 | 2.87 | 2.28 | 2.73 |
| Linear #2 | #1 | 2.48 | 2.52 | 1.70 | 1.97 |
| | #2 | 2.38 | 2.52 | 1.16 | 1.88 |
| | #3 | 3.03 | 3.03 | 2.70 | 3.02 |
| Sine | #1 | 2.76 | 2.57 | 1.77 | 2.09 |
| | #2 | 2.51 | 2.46 | 1.58 | 1.94 |
| | #3 | 2.37 | 2.38 | 1.54 | 1.79 |
| Sloth | #1 | 2.39 | 2.48 | 1.49 | 1.84 |
| | #2 | 2.37 | 2.47 | 0.87 | 1.73 |
| | #3 | 2.15 | 2.21 | 1.47 | 1.76 |

# Numerical results

## Results using cell marching:



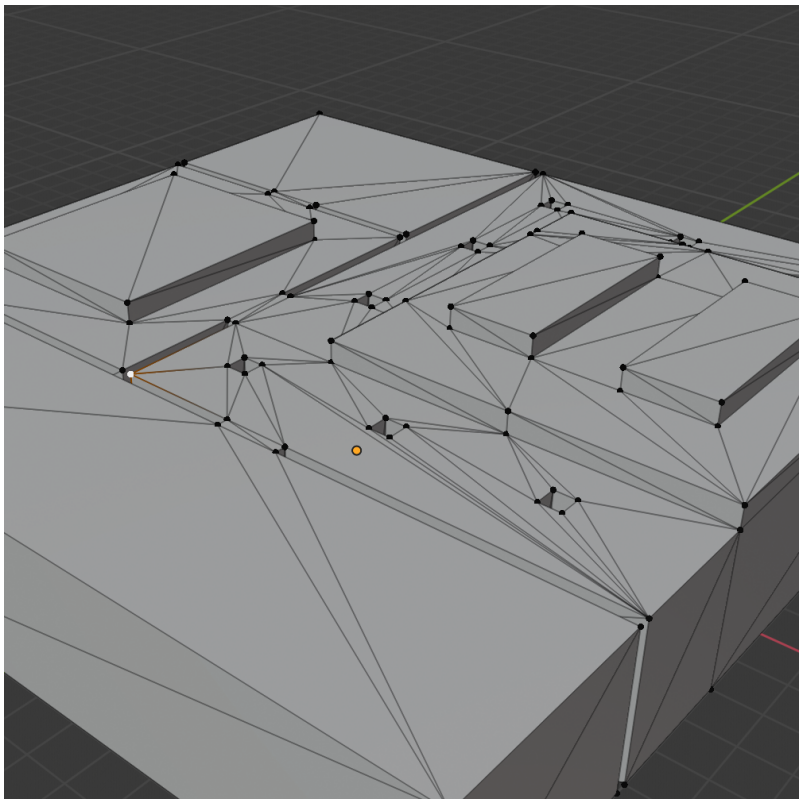

| | $\tau - T$ | $\tau_x - T_x$ | $\tau_y - T_y$ | $\tau_{xx} - T_{xx}$ | $\tau_{xy} - T_{xy}$ | $\tau_{yy} - T_{yy}$ |
|---|---|---|---|---|---|---|
| Constant | 3.09 | 3.11 | 3.11 | 2.01 | 2.05 | 2.01 |
| Linear #1 | 2.99 | 2.43 | 2.40 | 1.39 | 2.01 | 1.39 |
| Linear #2 | 2.10 | 1.76 | 1.72 | 0.77 | 1.25 | 0.77 |
| Sine | 2.91 | 1.80 | 1.89 | 0.73 | 1.31 | 0.80 |
| Sloth | 2.03 | 1.76 | 1.75 | 0.75 | 1.33 | 0.76 |

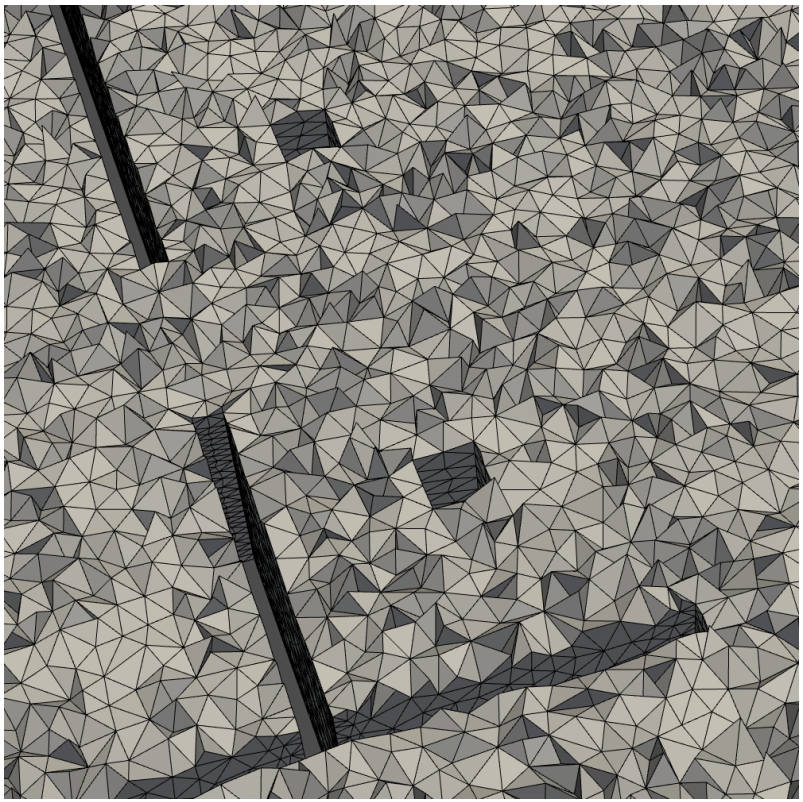
# Numerical geometric acoustics

(current work)

# Our goal (work in progress)

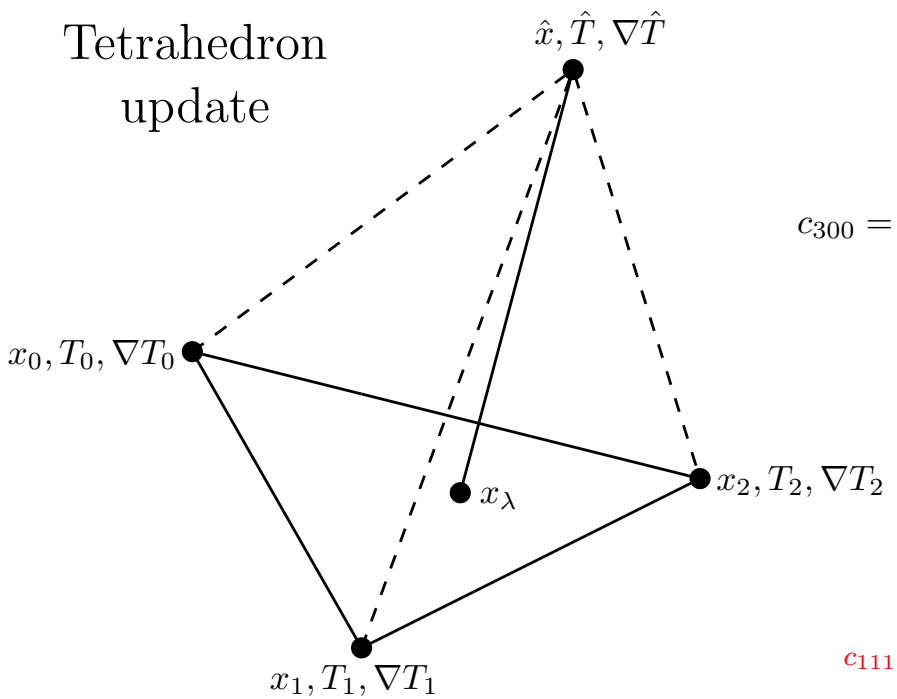
Develop JMM for solving the eikonal equation on a tetrahedron mesh and use this solver to compute acoustic parameters

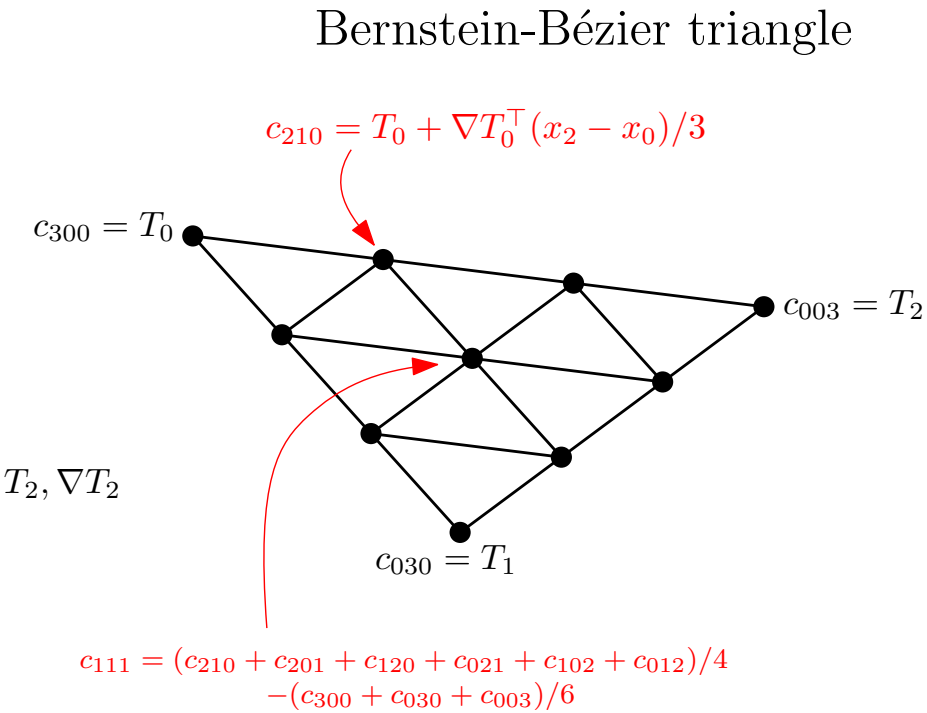




(A test scene made with Blender—mesh made using TetGen.)

# Extension to 3D: initial focus on $c \equiv 1$



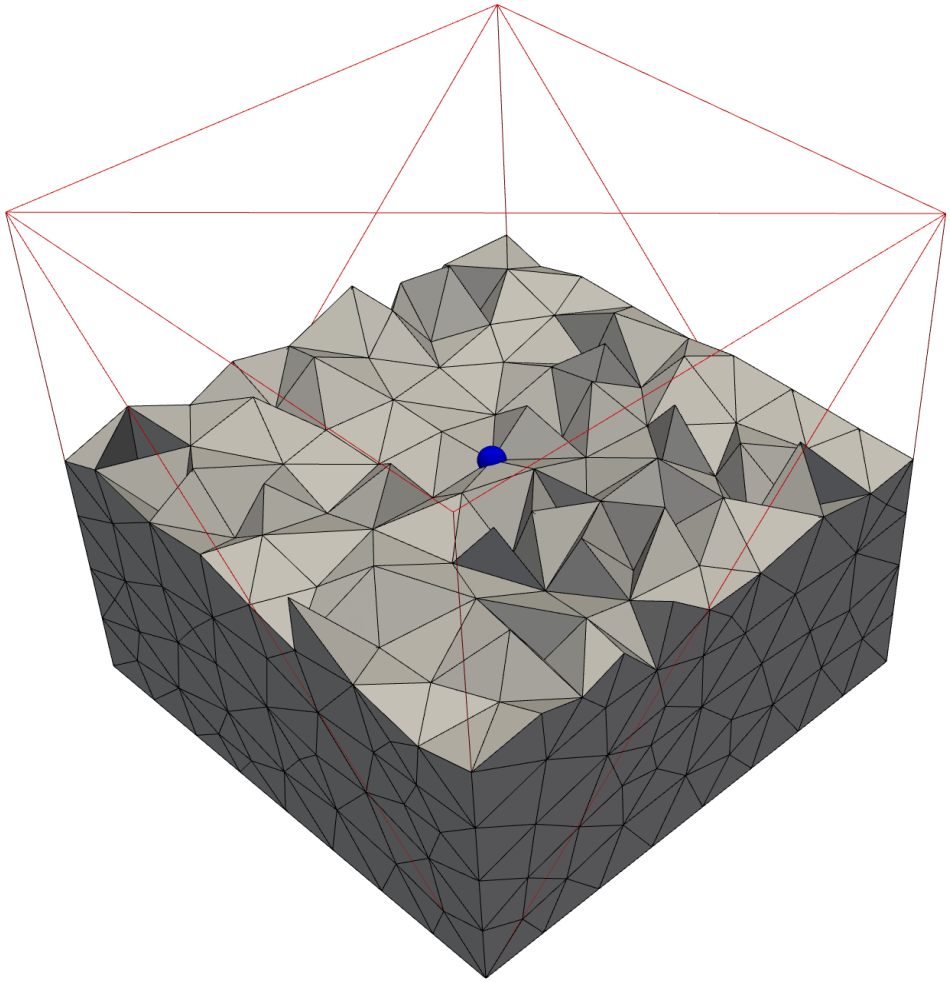

$$\hat{T} = \min_{\substack{\lambda_1 \geq 0, \lambda_2 \geq 0 \\ \lambda_1 + \lambda_2 \leq 1}} \left\{ T(x_\lambda) + \|\hat{x} - x_\lambda\| \right\}, \qquad \nabla \hat{T} = \frac{\hat{x} - x_{\lambda^*}}{\|\hat{x} - x_{\lambda^*}\|}$$

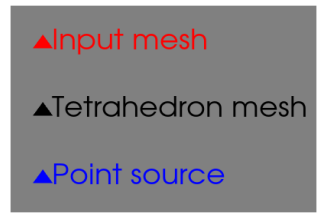
Accept update **only if** Lagrange multipliers are zero

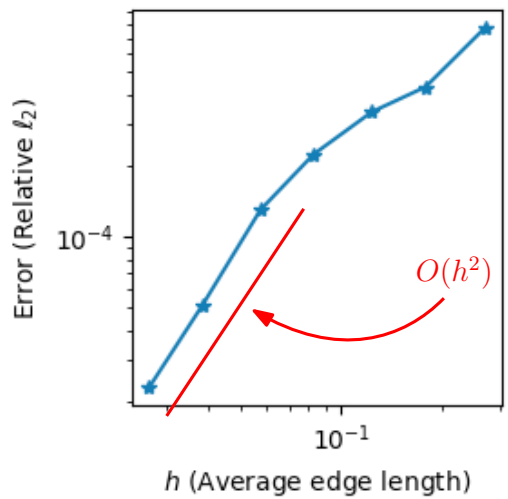
---

Source: "Triangular Bernstein-Bézier patches", Farin, 1986.

# Extension to 3D: box domain

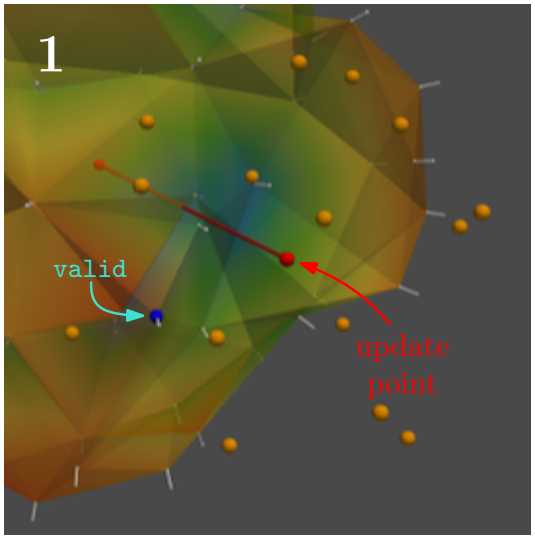


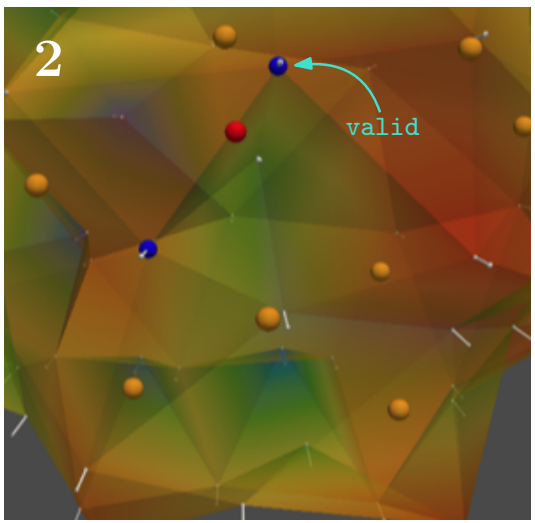


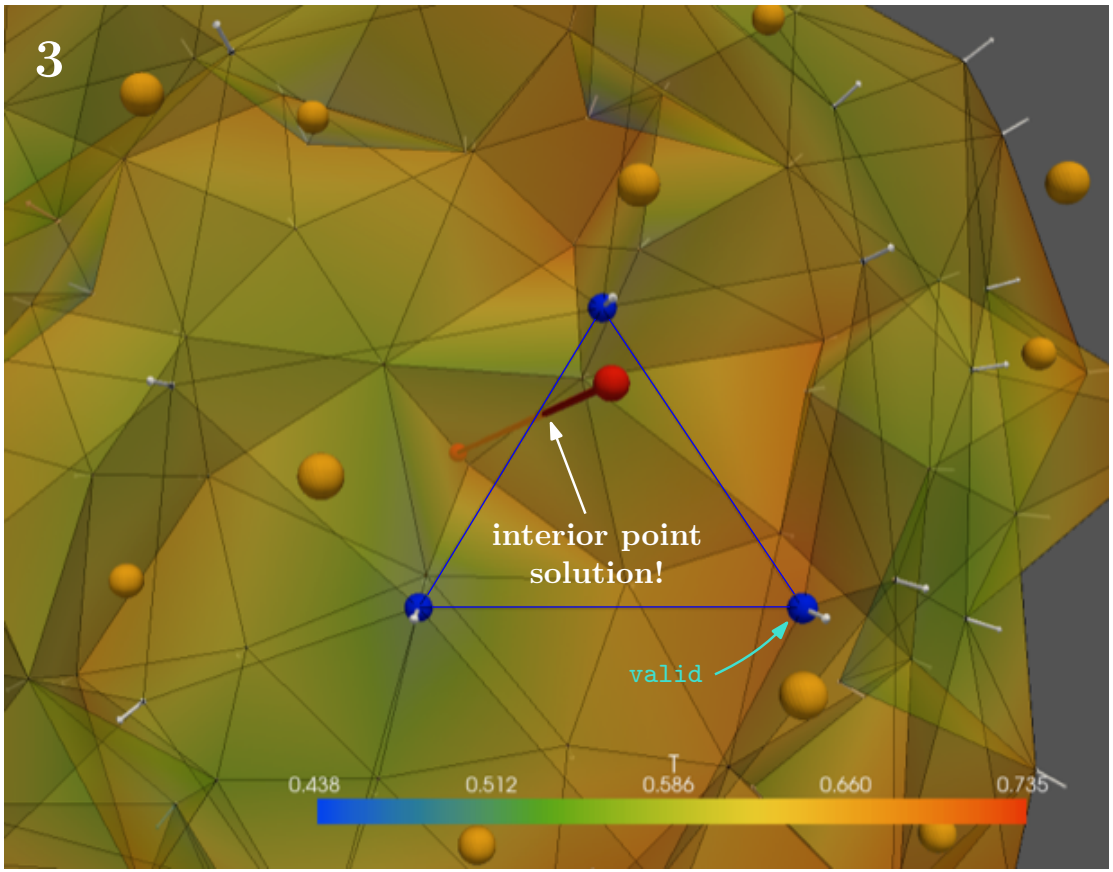




Domain is $\Omega = [-1, 1]^3$, place a point source at the origin so that $\tau(0) = 0$, use local factoring. Exact solution is simple: $\tau(x) = \|x\|$.

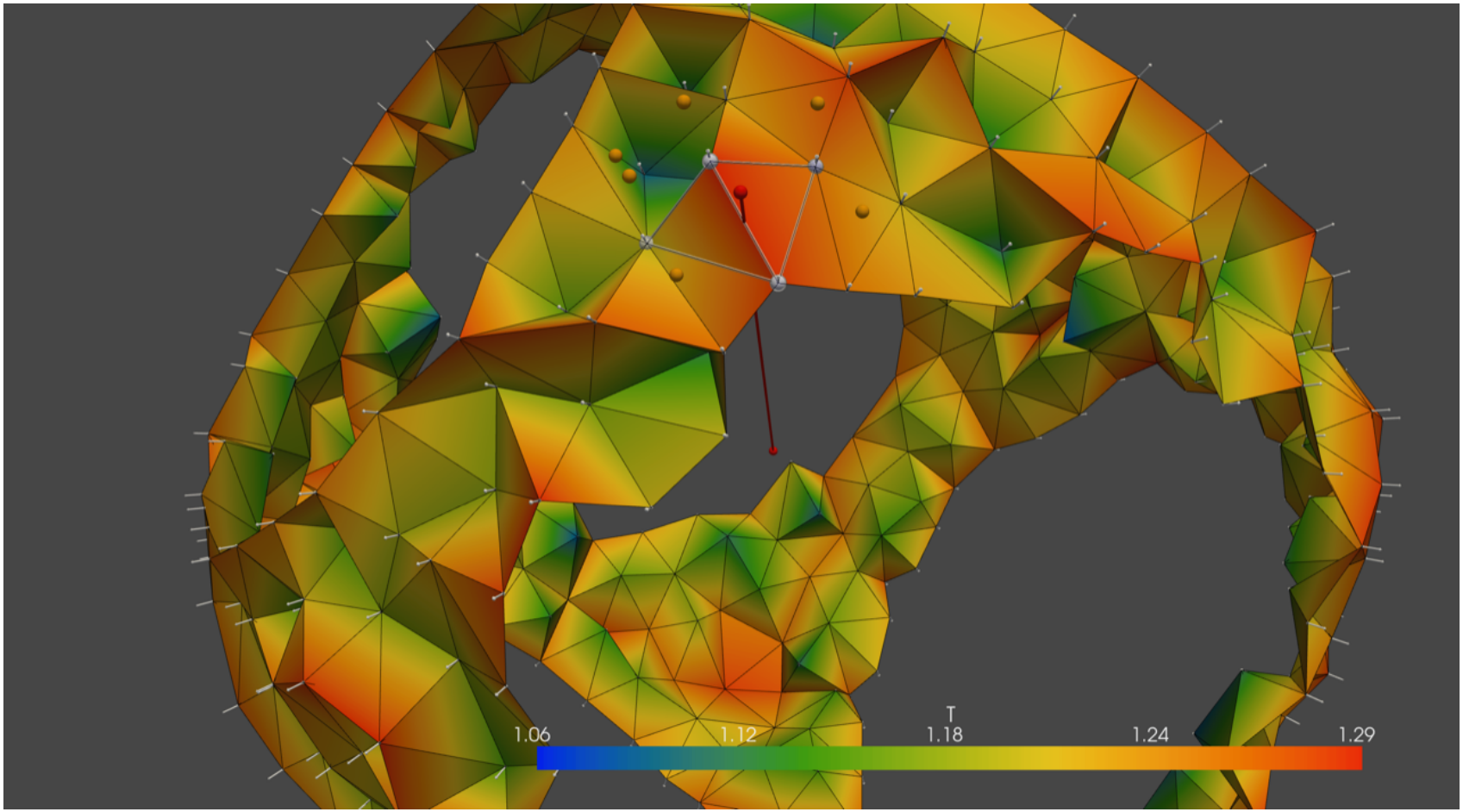
# Doing tetrahedron updates



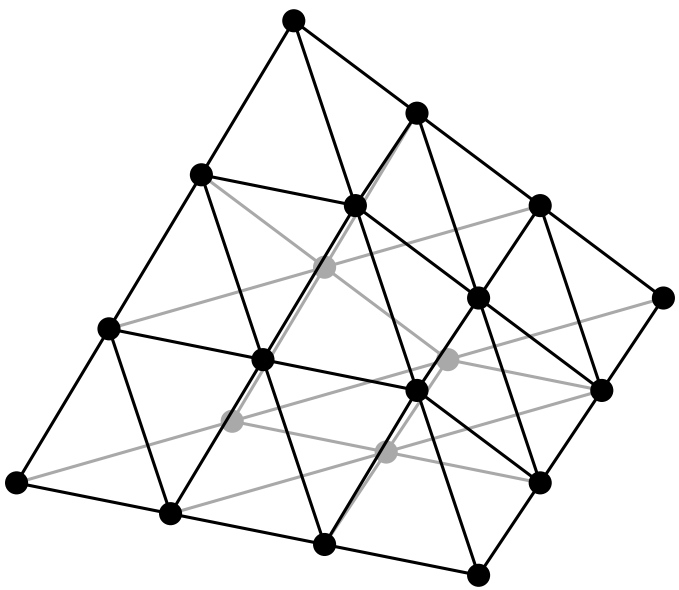



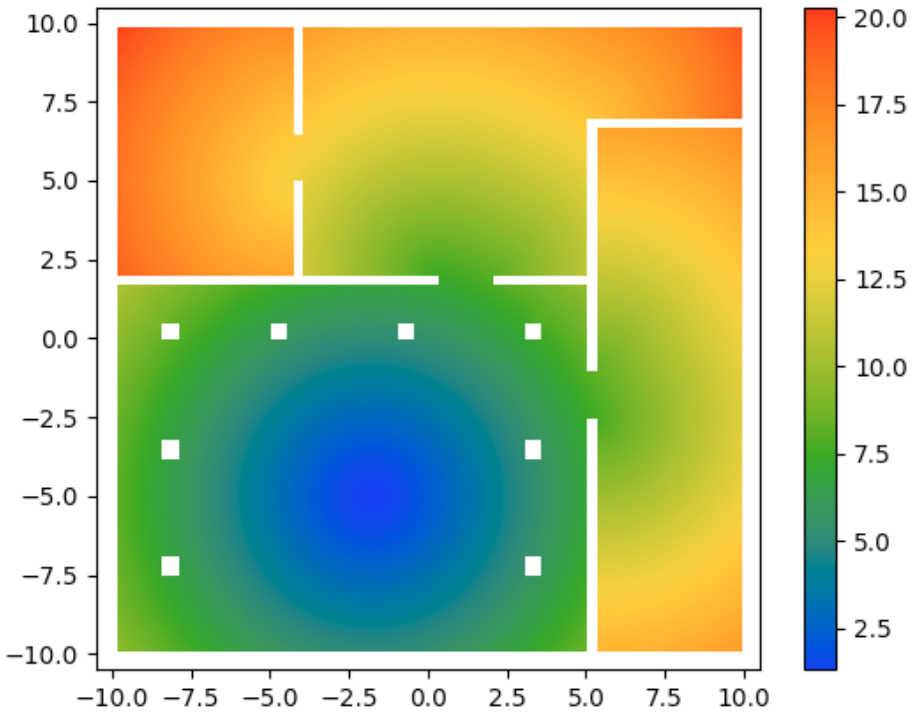
# Tetrahedron mesh challenges



# Test problem: small building with a few rooms

Build piecewise Hermite interpolant on tetrahedron mesh using Bernstein-Bézier *tetrahedra*, then regrid to regular grid





# Test problem: small building with a few rooms

*play movie*

# References

Samuel F. Potter, and Maria K. Cameron. "Jet Marching Methods for Solving the Eikonal Equation." *arXiv preprint arXiv:2009.05490* (2020).

Samuel F. Potter, Maria K. Cameron, and Ramani Duraiswami. "Precomputed Eikonal-based Sound Propagation." *In preparation.*